

[LOGO
ENSPY]

ÉCOLE NATIONALE SUPÉRIEURE POLYTECHNIQUE DE YAOUNDÉ

Département de Génie Informatique — 3^e Année

Introduction aux Réseaux — Semestre 2, 2025-2026

Business Core as a Service

BCaaS

« Une infrastructure générique de gestion des métiers
inspirée de l'ingénierie des protocoles réseaux »

Rédigé par :

Membre du groupe	Participation
ETCHOME Chris	_____ / 100
[Membre 2 — Nom Prénom]	_____ / 100
[Membre 3 — Nom Prénom]	_____ / 100
[Membre 4 — Nom Prénom]	_____ / 100
[Membre 5 — Nom Prénom]	_____ / 100
[Membre 6 — Nom Prénom]	_____ / 100

Sous la supervision de :

Pr. Dr Ing. Thomas Djotio Ndié

Enseignant — Introduction aux Réseaux

Résumé

Ce rapport présente la conception, l'implémentation et le déploiement de **BCaaS** (*Business Core as a Service*), une plateforme générique *multi-tenant* dédiée à la gestion standardisée des processus métier. Le projet s'inscrit dans le cadre du cours d'Introduction aux Réseaux de la troisième année de Génie Informatique de l'École Nationale Supérieure Polytechnique de Yaoundé.

L'idée centrale consiste à modéliser un système d'information métier en s'appuyant rigoureusement sur les concepts de l'ingénierie des protocoles réseaux : pile de couches, encapsulation, routage, adressage, qualité de service, et séparation des plans de contrôle et de données. Cette analogie formelle, loin d'être une simple métaphore pédagogique, structure l'ensemble des décisions architecturales : un *tenant* y joue le rôle d'un VLAN, une demande de service celui d'un paquet, et la facture celui d'un accusé de réception.

La solution livrée repose sur Spring Boot 3.5 / Java 21, PostgreSQL avec Liquibase, Spring Security avec authentification JWT et isolation multi-tenant par colonne `tenant_id`, et SpringDoc OpenAPI pour la documentation automatique de l'API. Le domaine compte quinze entités et l'API expose environ quatre-vingt-cinq points de terminaison REST validés par cent cinquante-deux tests automatisés. Un frontend Next.js (site vitrine et tableau de bord d'administration) complète l'ensemble. L'API est déployée sur Render et la documentation sur Vercel.

Mots-clés : Business Core, multi-tenant, protocoles réseaux, modèle OSI, encapsulation, Spring Boot, API REST, JWT, Next.js, architecture hexagonale.

Abstract

This report presents the design, implementation and deployment of **BCaaS** (*Business Core as a Service*), a generic *multi-tenant* platform dedicated to the standardized management of business processes. The project was carried out within the Introduction to Networks course of the third year of Computer Engineering at the National Advanced School of Engineering of Yaoundé.

The core idea is to model a business information system by rigorously relying on the concepts of network protocol engineering: layered stack, encapsulation, routing, addressing, quality of service, and the separation of control and data planes. This formal analogy is not a mere pedagogical metaphor; it drives every architectural decision: a *tenant* plays the role of a VLAN, a service request that of a packet, and an invoice that of an acknowledgement signal.

The delivered solution is built on Spring Boot 3.5 / Java 21, PostgreSQL with Liquibase, Spring Security with JWT authentication and `tenant_id`-based multi-tenant isolation, and SpringDoc OpenAPI for automatic API documentation. The domain comprises fifteen entities and the API exposes roughly eighty-five REST endpoints validated by one hundred and fifty-two automated tests. A Next.js frontend (public showcase site and administration dashboard) completes the system. The API is deployed on Render and the documentation on Vercel.

Keywords: Business Core, multi-tenant, network protocols, OSI model, encapsulation, Spring Boot, REST API, JWT, Next.js, hexagonal architecture.

Table des matières

Résumé	i
Abstract	ii
Liste des abréviations	ix
1 Introduction générale	1
1.1 Contexte du projet	1
1.2 Problème	1
1.3 Problématique	2
1.4 Objectifs	2
1.4.1 Objectif général	2
1.4.2 Objectifs spécifiques	2
1.5 Méthodologie	2
1.6 Structure du document	3
2 État de l’art	4
2.1 Modèles en couches : OSI et TCP/IP	4
2.2 Architectures multi-tenant	4
2.3 Le paradigme <i>as-a-Service</i> et le <i>Business as a Service</i>	5
2.4 Architecture hexagonale et Domain-Driven Design	5
2.5 Positionnement de BCaaS	5
3 Analogie formelle entre réseaux et métier	6
3.1 Table de correspondance réseau-métier	6
3.2 Émetteur, récepteur, protocole et message	7
3.3 Flux de traitement d’une requête	7
3.4 La pile protocolaire métier en cinq couches	7
3.4.1 Couche 1 — Infrastructure	8
3.4.2 Couche 2 — Transport & Messaging	8
3.4.3 Couche 3 — Tenant & Routing	8
3.4.4 Couche 4 — Context & Policy	8
3.4.5 Couche 5 — Business Capabilities	8
3.5 Le multi-tenant comme réseau virtuel (VLAN)	9
3.6 Plan de contrôle et plan de données	9

4	Analyse et spécification des besoins	10
4.1	Acteurs du système	10
4.2	Besoins fonctionnels	10
4.3	Besoins non fonctionnels	12
4.4	Diagramme de cas d'utilisation	12
4.5	Diagramme de contexte	13
5	Modélisation mathématique	14
5.1	Ensembles fondamentaux	14
5.1.1	Décomposition des acteurs	14
5.1.2	Attributs des entités principales	14
5.2	Variables de décision et fonction objectif	14
5.2.1	Variables	15
5.2.2	Objectif multi-critère	15
5.3	Contraintes du système	15
5.3.1	Contraintes de cardinalité	15
5.3.2	Contraintes métier	15
5.3.3	Contrainte de workflow : machine à états	16
5.4	Algorithmes et complexité	16
5.4.1	Recherche des fournisseurs d'un métier	16
5.4.2	Recherche par filtres combinés	16
5.5	Indicateurs et métriques (KPIs)	17
6	Conception architecturale	18
6.1	Architecture hexagonale et DDD	18
6.2	Monolithe modulaire « microservices-ready »	18
6.3	Diagramme de packages	19
6.4	Diagramme de déploiement	19
7	Conception détaillée des données	21
7.1	Modèle Conceptuel de Données (MCD)	21
7.2	Dictionnaire des entités	21
7.3	Modèle Logique de Données (MLD)	22
7.4	Diagramme de classes métier	23
7.5	Contraintes d'intégrité	23
7.6	Machines à états	24
8	Implémentation technique	25
8.1	Pile technologique	25
8.2	Structure du projet	25
8.3	Multi-tenant et isolation	26
8.4	Sécurité : JWT et RBAC	27
8.5	Persistance et migrations	27
8.6	Temps réel et événementiel	27

8.7	Documentation et conteneurisation	27
9	Protocole d'échange pour un cas métier concret	28
9.1	Cas choisi : création d'un Business et ajout au catalogue	28
9.2	Protocole : création d'un Business	28
9.3	Protocole : ajout au catalogue de services	29
9.4	Diagramme de séquence	29
10	Traçabilité et observabilité	31
10.1	Piste d'audit	31
10.2	Tracing : <code>traceId</code> et <code>correlationId</code>	31
10.3	Analytique et KPIs	31
10.4	Logs structurés (proposition)	32
11	Frontend et déploiement	33
11.1	Architecture du frontend	33
11.2	Choix techniques	33
11.3	Pages principales	33
11.4	Déploiement	33
11.5	Captures d'écran	34
12	Tests et validation	35
12.1	Stratégie de test	35
12.2	Critères d'acceptation	35
12.3	Scénario de validation bout-en-bout	35
13	Gestion de projet	37
13.1	Méthodologie	37
13.2	Organisation des sprints	37
13.3	Répartition des tâches et participation	37
13.4	Outils et gestion de version	38
14	Perspectives d'amélioration	39
14.1	Court terme : finalisation des briques posées	39
14.2	Moyen terme : robustesse et qualité de service	39
14.3	Long terme : montée en charge et nouvelles contraintes	39
15	Conclusion générale	41
15.1	Bilan	41
15.2	Limites	41
15.3	Apports personnels	41
	Bibliographie	43
	Glossaire	44

A	Annexes	46
A.1	Liste des endpoints REST	46
A.2	Exemple de log structuré	47
A.3	Capture Swagger UI	47
A.4	Extraits de code significatifs	47

Table des figures

3.1	Flux bout-en-bout d'une requête dans BCaaS — analogie OSI	7
3.2	Pile protocolaire métier de BCaaS et correspondance avec le modèle OSI	8
4.1	Diagramme de cas d'utilisation de BCaaS	12
4.2	Diagramme de contexte du système	13
6.1	Architecture logique de BCaaS	19
6.2	Diagramme de packages de l'architecture du système	19
6.3	Diagramme de déploiement de BCaaS	20
7.1	Modèle Conceptuel de Données de BCaaS	21
7.2	Modèle Logique de Données de BCaaS	23
7.3	Diagramme de classes métier de BCaaS	23
7.4	Machines à états de ServiceRequest et Invoice	24
9.1	Diagramme de séquence du cas métier « Business + Catalogue »	30
11.1	Page d'accueil du site vitrine	34
11.2	Tableau de bord d'administration	34
A.1	Documentation interactive Swagger UI	47

Liste des tableaux

2.1	Comparaison des stratégies d'isolation multi-tenant	4
3.1	Correspondance formelle entre concepts réseaux et concepts métier	6
4.1	Acteurs du système	10
4.2	Besoins fonctionnels — Identités (Tenant, User, Actor)	11
4.3	Besoins fonctionnels — Métiers (Business, Rule, Catalogue, Resource, Media, Portfolio)	11
4.4	Besoins fonctionnels — Transactions (ServiceRequest, Invoice, Message, Notification)	11
4.5	Besoins non fonctionnels et statut dans le MVP	12
5.1	Transitions valides de la machine à états ServiceRequest	16
7.1	Entités du modèle de données — Business Core	22
7.2	Principales contraintes d'intégrité	24
8.1	Pile technologique de BCaaS	25
10.1	Métriques de BCaaS et leur analogie réseau	32
11.1	Technologies du frontend	33
12.1	Critères d'acceptation	35
13.1	Découpage en sprints	37
13.2	Répartition de la participation	38
A.1	Synthèse des endpoints REST	46

Liste des abréviations

Sigle	Signification
BCaaS	Business Core as a Service
BaaS	Business as a Service
API	Application Programming Interface
REST	Representational State Transfer
JWT	JSON Web Token
RBAC	Role-Based Access Control
DDD	Domain-Driven Design
DTO	Data Transfer Object
FSM	Finite State Machine (machine à états finis)
OSI	Open Systems Interconnection
TCP/IP	Transmission Control Protocol / Internet Protocol
VLAN	Virtual Local Area Network
SDN	Software-Defined Networking
QoS	Quality of Service
SLA	Service-Level Agreement
MCD	Modèle Conceptuel de Données
MLD	Modèle Logique de Données
UML	Unified Modeling Language
PWA	Progressive Web App
SSR	Server-Side Rendering
CdS	Consommateur de Service
FdS	Fournisseur de Service
KPI	Key Performance Indicator
RLS	Row-Level Security
UUID	Universally Unique Identifier

CHAPITRE 1

Introduction générale

1.1 Contexte du projet

Dans le domaine du génie logiciel, chaque nouveau projet métier — qu’il s’agisse d’une pharmacie, d’une agence immobilière, d’un cabinet médical, d’une auto-école ou d’une école — reconstruit invariablement les mêmes fondations : gestion des acteurs, des ressources, des règles, des workflows, des notifications et de la traçabilité. Cette duplication systématique représente un gaspillage considérable en temps de développement, en coût de maintenance et en cohérence architecturale.

Parallèlement, l’ingénierie des réseaux a résolu depuis des décennies un problème structurellement similaire : faire communiquer de façon fiable, sécurisée et interopérable des entités hétérogènes sur une infrastructure partagée. Le modèle OSI et la suite TCP/IP proposent une décomposition en couches aux responsabilités clairement délimitées, un mécanisme d’encapsulation des données, un adressage précis des destinataires, une qualité de service différenciée, et une séparation nette entre plan de contrôle et plan de données.

Le présent projet s’inscrit dans le cours d’Introduction aux Réseaux de la 3^e année de Génie Informatique, dont l’objectif pédagogique est précisément d’établir un *lien formel* entre les concepts du domaine des réseaux et ceux d’un domaine métier. BCaaS constitue le *noyau générique* (*Business Core*) sur lequel les autres instances métier de la promotion (pharmacie, immobilier, tourisme, médical, etc.) peuvent se greffer.

1.2 Problème

Le problème adressé est celui de la **ré-implémentation perpétuelle des fondations métier**. Concrètement :

- la fragmentation des outils de gestion entre métiers, sans interopérabilité ;
- l’absence de standardisation des concepts récurrents (acteur, service, ressource, règle métier, facture) ;
- la difficulté de mise en relation entre un consommateur de service (CdS) et un fournisseur de service (FdS) ;
- le manque de traçabilité du cycle de vie des prestations ;
- la gestion manuelle et non auditée de la facturation.

1.3 Problématique

Question directrice

Comment concevoir un noyau métier générique, réutilisable et configurable qui permette à n'importe quelle instance métier de fonctionner sans reconstruire les fondations, en s'appuyant formellement sur les concepts de l'ingénierie des protocoles réseaux ?

Cette question directrice se décline en sous-questions : quels concepts réseaux peut-on mettre en correspondance avec quels composants métier ? Comment garantir l'isolation des données de clients distincts sur une infrastructure partagée, à la manière d'un VLAN ? Comment formaliser le « protocole » d'échange d'un cas métier concret ?

1.4 Objectifs

1.4.1 Objectif général

Concevoir et réaliser une plateforme générique multi-tenant de gestion des métiers, architecturée selon une analogie rigoureuse avec la pile protocolaire réseau.

1.4.2 Objectifs spécifiques

1. Établir une analogie formelle et rigoureuse entre les concepts réseaux (couches OSI, encapsulation, routage, QoS, multi-tenant) et les composants d'une architecture métier générique.
2. Concevoir une architecture modulaire multi-tenant, lisible à travers une pile protocolaire métier en cinq couches.
3. Implémenter une API REST de provisionnement de tenants reposant sur un modèle de données générique configurable.
4. Sécuriser les échanges par authentification JWT et contrôle d'accès basé sur les rôles (RBAC), avec isolation stricte par tenant.
5. Documenter et exposer l'API via Swagger UI, et déployer la solution (API et documentation) sur le cloud.
6. Développer un frontend Next.js comprenant un site vitrine et un tableau de bord d'administration.

1.5 Méthodologie

Le projet a été conduit de manière incrémentale, organisée en *sprints* successifs assortis de critères de priorité (P0 : indispensable ; P1 : important). L'architecture retenue est une **architecture hexagonale** (ports et adaptateurs) combinée au **Domain-Driven Design** (DDD), afin de préserver l'indépendance du domaine métier vis-à-vis des détails techniques. Chaque

étape a fait l'objet d'une validation explicite (modélisation Merise, diagrammes UML, audits de code, tests automatisés) avant de passer à la suivante.

1.6 Structure du document

Le rapport est organisé comme suit. Le chapitre 2 dresse l'état de l'art. Le chapitre 3 formalise l'analogie réseau-métier, cœur conceptuel du projet, et y présente la pile protocolaire en cinq couches. Le chapitre 4 spécifie les besoins fonctionnels et non fonctionnels. Le chapitre 5 propose une modélisation mathématique du système. Les chapitres 6 et 7 détaillent la conception architecturale et la conception des données. Le chapitre 8 décrit l'implémentation technique, le chapitre 9 le protocole d'échange d'un cas métier, et le chapitre 10 la traçabilité et l'observabilité. Le chapitre 11 traite du frontend et du déploiement, le chapitre 12 des tests et de la validation, et le chapitre 13 de la gestion de projet. Le chapitre 14 expose les perspectives d'amélioration, avant la conclusion générale (chapitre 15).

CHAPITRE 2

État de l'art

Ce chapitre situe BCaaS dans son paysage scientifique et technique. Il passe en revue les modèles en couches issus des réseaux, les stratégies d'architecture multi-tenant, le paradigme *as-a-Service*, ainsi que les principes d'architecture hexagonale et de Domain-Driven Design, avant de positionner notre contribution.

2.1 Modèles en couches : OSI et TCP/IP

Le modèle de référence OSI (*Open Systems Interconnection*), normalisé par l'ISO [2], décompose la communication en sept couches aux responsabilités disjointes (physique, liaison, réseau, transport, session, présentation, application). Chaque couche n'expose qu'un service à la couche supérieure et s'appuie sur la couche inférieure : c'est le principe d'*encapsulation*, où chaque niveau ajoute son en-tête à l'unité de données reçue. La suite TCP/IP, plus pragmatique, condense ces niveaux mais conserve les mêmes idées directrices : adressage, routage, fiabilité et contrôle de flux.

Ces modèles présentent un intérêt méthodologique qui dépasse les réseaux : la *séparation des préoccupations* par couches, l'encapsulation progressive du contexte et l'adressage explicite des destinataires sont des principes de conception logicielle universels. C'est cette transposition que BCaaS exploite.

2.2 Architectures multi-tenant

Une application *multi-tenant* sert plusieurs clients (*tenants*) sur une infrastructure mutualisée tout en garantissant l'isolation de leurs données. Trois stratégies d'isolation sont classiquement distinguées [9], présentées au tableau 2.1.

Table 2.1: *Comparaison des stratégies d'isolation multi-tenant*

Stratégie	Isolation	Performance	Complexité / Coût
Colonne <code>tenant_id</code> (schéma partagé)	Logique	Bonne (index)	Faible
Schéma PostgreSQL séparé	Physique	Très bonne	Moyenne
Base de données séparée	Totale	Variable	Élevée

BCaaS adopte la première stratégie (schéma partagé, colonne `tenant_id`), la plus simple à opérer pour un MVP, tout en gardant les autres comme évolutions possibles (chapitre 14). L'analogie réseau est immédiate : le partage d'infrastructure avec étiquetage logique des flux correspond exactement au principe du VLAN.

2.3 Le paradigme *as-a-Service* et le *Business as a Service*

Les modèles *as-a-Service* (IaaS, PaaS, SaaS) ont popularisé l'idée de fournir une capacité réutilisable via une interface standardisée, déchargeant le consommateur de la construction de l'infrastructure sous-jacente. Le *Business as a Service* (BaaS) transpose cette logique au niveau métier : il s'agit d'exposer, comme un service générique, les briques récurrentes de tout système d'information professionnel (acteurs, métiers, services, ressources, règles, facturation). BCaaS est une mise en œuvre de ce paradigme, pensée comme socle commun à plusieurs instances métier.

2.4 Architecture hexagonale et Domain-Driven Design

L'architecture hexagonale (*ports and adapters*), proposée par Cockburn [3], isole le cœur métier des détails techniques (base de données, interface web, messagerie) au moyen de *ports* (interfaces) et d'*adaptateurs* (implémentations). Le Domain-Driven Design d'Evans [4] fournit le vocabulaire de modélisation associé : entités, agrégats, services de domaine, langage omniprésent. Ensemble, ces approches permettent un monolithe *modulaire* dont les modules pourront, à terme, être extraits en microservices sans réécriture du domaine.

2.5 Positionnement de BCaaS

La littérature fournit séparément les modèles en couches, les stratégies multi-tenant et les principes DDD/hexagonal. L'originalité de BCaaS n'est pas d'inventer l'un de ces éléments, mais de les **unifier sous une grille de lecture réseau cohérente** : la pile métier est interprétée comme une pile protocolaire, l'isolation multi-tenant comme un VLAN, la facture comme un accusé de réception, et l'administration comme un plan de contrôle SDN. Cette cohérence conceptuelle, détaillée au chapitre suivant, constitue l'apport central du travail.

CHAPITRE 3

Analogie formelle entre réseaux et métier

Ce chapitre constitue le cœur conceptuel du projet. Il établit une correspondance systématique entre les concepts de l'ingénierie des protocoles réseaux et les composants de BCaaS, puis présente la pile protocolaire métier en cinq couches qui en découle.

3.1 Table de correspondance réseau–métier

Le tableau 3.1 formalise la mise en correspondance. Chaque concept réseau y trouve un équivalent métier précis dans BCaaS, ce qui confère à l'architecture sa cohérence d'ensemble.

Table 3.1: *Correspondance formelle entre concepts réseaux et concepts métier*

Concept réseau	Définition réseau	Équivalent métier dans BCaaS
Adresse IP	Identifiant unique d'un nœud	<code>tenant_id</code> + domaine + service
En-tête de paquet	Métadonnées d'acheminement	<code>tenant_id</code> , <code>actor_id</code> , <code>trace_id</code>
Payload / SDU	Données applicatives	Corps de la requête métier (JSON)
Encapsulation	Ajout d'en-têtes à chaque couche	Enrichissement du contexte au passage des couches
Routage	Sélection du chemin vers la destination	Résolution <code>tenant</code> → service → instance
NAT / Proxy	Traduction d'adresses	API Gateway : réécriture et normalisation
VLAN	Isolation logique sur réseau partagé	Isolation par <code>tenant_id</code> sur schéma partagé
TCP (fiable)	Transport avec accusé de réception	REST synchrone avec retry et idempotence
QoS	Priorité et garanties de bande passante	SLA par tenant, priorité de transaction
TTL	Durée de vie d'un paquet	Expiration du token JWT
Control plane	Gestion des routes (BGP, OSPF)	Admin : configuration tenants, règles, SLA
Data plane	Acheminement des paquets	Exécution des transactions métier
Firewall	Filtrage de paquets	Spring Security, contrôle d'accès API

3.2 Émetteur, récepteur, protocole et message

Le sujet du cours invite à identifier explicitement les rôles de communication. Dans BCaaS, ils se traduisent ainsi :

- **Émetteur (source de la requête)** : le *Consommateur de Service* (CdS), un *Actor* qui émet une *ServiceRequest*.
- **Récepteur (destination de la requête, source de la réponse)** : le *Fournisseur de Service* (FdS), un *Actor* qui accepte, exécute et finalise la prestation.
- **Protocole (règles du métier)** : les *BusinessRule*, paires clé-valeur qui gouvernent le comportement d'un métier donné.
- **Message / requête** : la *ServiceRequest*, porteuse d'un *traceId* et d'un *correlationId* jouant le rôle d'identifiants de flux.
- **Accusé de réception (ACK)** : l'*Invoice*, générée automatiquement à la finalisation et confirmant la bonne exécution.
- **Canal** : l'API REST sécurisée par JWT.

3.3 Flux de traitement d'une requête

Le traitement d'une requête suit le modèle OSI descendant (émission) puis ascendant (réception) : le payload métier est progressivement enrichi de son en-tête tenant, de son contexte d'exécution, puis de ses permissions et de son niveau de SLA, avant d'atteindre le service métier qui produit la réponse et publie les événements associés (notification, audit). La figure 3.1 illustre ce flux bout-en-bout.

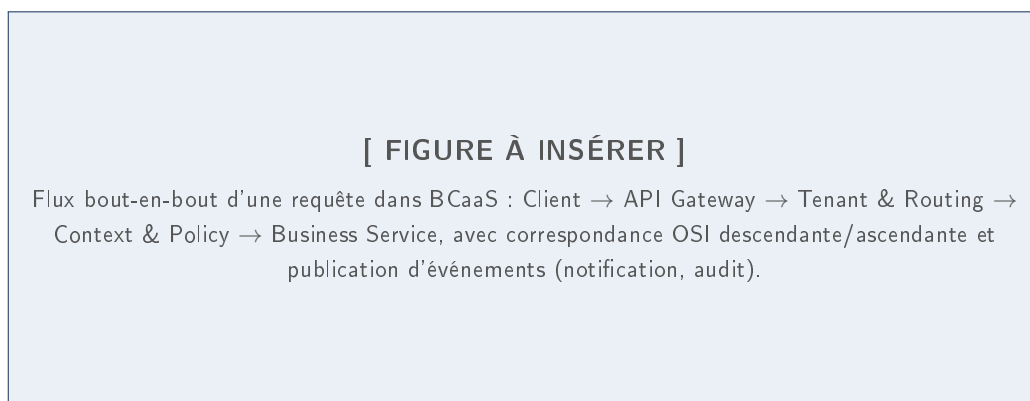


Figure 3.1: Flux bout-en-bout d'une requête dans BCaaS — analogie OSI

3.4 La pile protocolaire métier en cinq couches

De l'analogie découle une décomposition de BCaaS en cinq couches protocolaires, en correspondance directe avec le modèle OSI (figure 3.2). Chaque couche n'expose qu'un service bien défini à la couche supérieure.

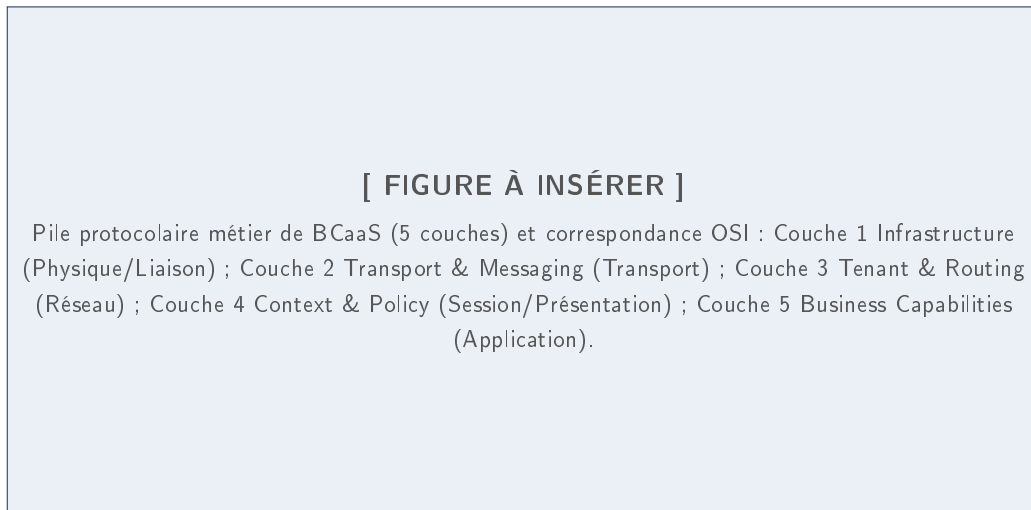


Figure 3.2: Pile protocolaire métier de BCaaS et correspondance avec le modèle OSI

3.4.1 Couche 1 — Infrastructure

Substrat technique : base de données PostgreSQL avec migrations Liquibase, cache Redis et bus de messages. Elle correspond aux couches physique et liaison : elle transporte les données sans en connaître la signification.

3.4.2 Couche 2 — Transport & Messaging

Analogue à la couche transport (TCP/UDP). Elle gère le mode d'échange (synchrone via REST, ou asynchrone via le bus d'événements), les stratégies de retry, les timeouts et l'idempotence. Le choix *REST* $\equiv$ *TCP* (fiable, ordonné) ou *événement* $\equiv$ *UDP* (rapide, au mieux) est explicite dans la conception.

3.4.3 Couche 3 — Tenant & Routing

Analogue à la couche réseau. Le *Tenant Resolver* joue le rôle d'un routeur : à partir de l'en-tête de la requête (le claim `tenantId` du JWT), il détermine le contexte d'exécution et oriente la requête vers le bon service. C'est ici que réside la logique de découverte et d'isolation.

3.4.4 Couche 4 — Context & Policy

Analogue aux couches session et présentation. Cette couche valide le token JWT, décode les rôles et permissions, applique les politiques de sécurité propres au tenant et détermine le niveau de service. C'est ici qu'est construit l'« en-tête métier » complet de la requête.

3.4.5 Couche 5 — Business Capabilities

Couche applicative. Elle contient les services métier génériques (acteurs, métiers, services, ressources, règles, factures, audit), qui opèrent sur des données configurables par tenant sans connaissance des couches inférieures.

3.5 Le multi-tenant comme réseau virtuel (VLAN)

Analogie : VLAN ↔ Tenant

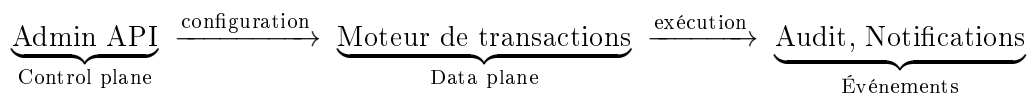
Dans un réseau, un VLAN isole logiquement plusieurs groupes de machines sur la même infrastructure physique ; chaque trame porte un *tag* (802.1Q) identifiant son appartenance. Dans BCaaS, chaque requête porte un `tenant_id` qui isole les données, les règles et les workflows d'un client sur l'infrastructure partagée.

$\text{tag VLAN} \equiv \text{tenant_id}, \quad \text{réseau physique} \equiv \text{infrastructure partagée}.$

3.6 Plan de contrôle et plan de données

Analogie : SDN ↔ Administration BCaaS

Dans les réseaux définis par logiciel (SDN), le *control plane* configure de façon centralisée les règles de routage, tandis que le *data plane* achemine les paquets. Dans BCaaS, l'interface d'administration (control plane) configure tenants, règles métier, catalogues et niveaux de service ; le moteur d'exécution (data plane) traite les transactions en appliquant ces configurations, et émet les événements d'audit et de notification.



CHAPITRE 4

Analyse et spécification des besoins

4.1 Acteurs du système

Le tableau 4.1 décrit les acteurs. Sur le plan métier, une personne physique est modélisée par un **User** rattaché à un tenant ; elle joue un ou plusieurs rôles via l'entité **Actor** (consommateur et/ou fournisseur). Le contrôle d'accès applicatif repose sur un ensemble de rôles (**USER**, **ADMIN**, **PROVIDER**, **CONSUMER**).

Table 4.1: *Acteurs du système*

Acteur	Description	Rôles possibles
Tenant	Organisation cliente ; contexte d'isolation des données et configurations	—
User	Identité authentifiable rattachée à un tenant	USER, ADMIN, PROVIDER, CONSUMER
Consumer (CdS)	Acteur demandeur de services	CONSUMER
Provider (FdS)	Acteur fournisseur de services	PROVIDER
Admin	Administrateur (plan de contrôle)	ADMIN

Remarque. Le **Tenant** est un contexte de configuration et d'isolation, non un acteur de cas d'utilisation ; il n'apparaît donc pas dans le diagramme de cas d'utilisation au même titre que les utilisateurs humains.

4.2 Besoins fonctionnels

Les besoins fonctionnels sont identifiés par un code **F-XXX-NNN**. Les tableaux 4.2 à 4.4 les regroupent par domaine.

Table 4.2: *Besoins fonctionnels — Identités (Tenant, User, Actor)*

ID	Fonctionnalité	Contraintes
F-TEN-001	Provisionner un tenant (inscription publique)	Nom et domaine obligatoires, <code>tenant_id</code> UUID généré
F-USR-001	Inscrire un utilisateur (register)	Email unique, mot de passe chiffré (BCrypt)
F-USR-002	Authentifier (login)	Retourne un JWT portant rôles et <code>tenantId</code>
F-USR-003	Gérer les rôles d'un utilisateur	Réservé au rôle ADMIN
F-ACT-001	Créer un acteur lié à un utilisateur	Un utilisateur peut avoir plusieurs acteurs
F-ACT-002	Filtrer les acteurs par rôle	—

Table 4.3: *Besoins fonctionnels — Métiers (Business, Rule, Catalogue, Resource, Media, Portfolio)*

ID	Fonctionnalité	Contraintes
F-BUS-001	Créer / modifier un métier	Nom et domaine obligatoires ; rattaché au tenant
F-BUS-002	Rechercher par domaine	—
F-BRU-001	Définir une règle métier (clé-valeur)	Rattachée à un Business
F-SCA-001	Créer un service au catalogue	Prix de base + devise validée
F-SCA-002	Filtrer les services disponibles	<code>isAvailable</code>
F-RES-001	Gérer une ressource (stock)	Quantité ≥ 0
F-MED-001	Associer un média à un métier	Type + URL
F-POR-001	Gérer un portfolio de compétences	1 portfolio par acteur ; relation N:M avec Business

Table 4.4: *Besoins fonctionnels — Transactions (ServiceRequest, Invoice, Message, Notification)*

ID	Fonctionnalité	Contraintes
F-SRE-001	Créer une demande de service (CdS)	Consumer, provider et service du catalogue requis
F-SRE-002	Faire évoluer le statut	FSM : accept / start / fulfill / cancel
F-SRE-003	Suivre par statut / acteur	—
F-INV-001	Générer une facture	Automatique à la finalisation (ACK)
F-INV-002	Payer / annuler une facture	FSM : pay / cancel
F-MSG-001	Échanger des messages sur une demande	Temps réel (publication/abonnement)
F-NOT-001	Notifier un utilisateur	Compteur de non-lus, marquage lu
F-AUD-001	Tracer les événements (audit)	Par <code>traceId</code> , acteur, tenant
F-ANA-001	Calculer les KPIs d'un tenant	—

4.3 Besoins non fonctionnels

Le sujet impose un ensemble exigeant de contraintes non fonctionnelles. Le tableau 4.5 en précise le statut dans le présent MVP, dans une logique de transparence : ce qui n'est pas pleinement réalisé est repris comme perspective au chapitre 14.

Table 4.5: *Besoins non fonctionnels et statut dans le MVP*

Besoin	Description	Statut MVP
Multi-tenant	Isolation des données par tenant sur infrastructure partagée	Réalisé
Sécurité	Authentification JWT, mots de passe chiffrés, RBAC	Réalisé
Multi-devises	Support XAF, XOF, NGN, KES, GHS, USD, EUR, GBP	Réalisé
Documentation API	Spécification OpenAPI / Swagger UI	Réalisé
Conteneurisation	Docker / docker-compose	Réalisé
Modularité (DDD / hexagonal)	Monolithe modulaire, microservices-ready	Réalisé
Temps réel	Messagerie instantanée sur les demandes	Partiel (Redis pub/sub)
Événementiel	Bus d'événements (notifications, audit)	Partiel (infra posée)
Géo-référencement	PostGIS, géolocalisation native	Perspective
Offline-first / PWA	Application installable et hors-ligne	Perspective
Internationalisation	i18n web et mobile	Perspective
Réactif	WebFlux + R2DBC non bloquant	Perspective
2FA	Double authentification	Perspective
Mobile natif	React Native	Perspective

4.4 Diagramme de cas d'utilisation

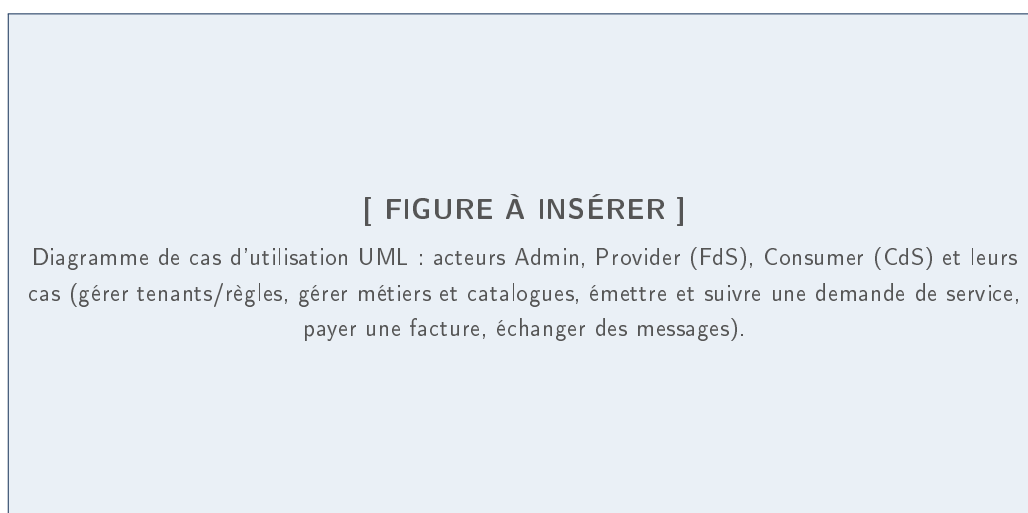


Figure 4.1: *Diagramme de cas d'utilisation de BCaaS*

4.5 Diagramme de contexte

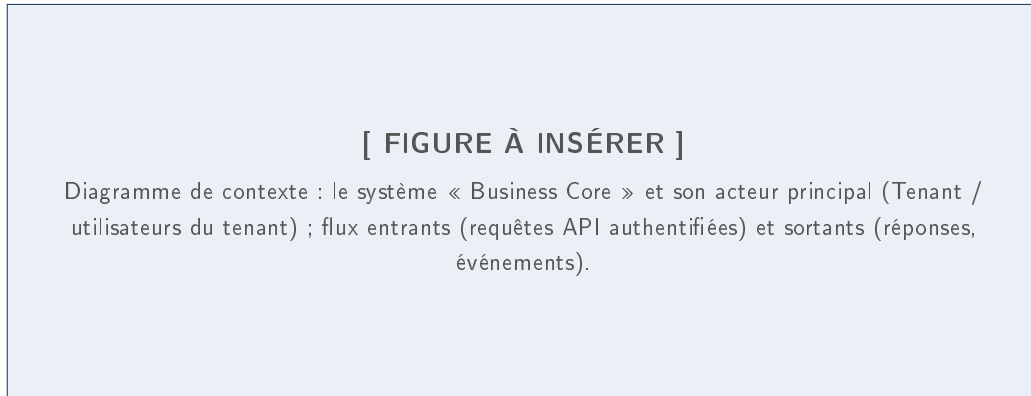


Figure 4.2: *Diagramme de contexte du système*

CHAPITRE 5

Modélisation mathématique

Ce chapitre propose une formalisation ensembliste du système, de ses contraintes et de ses indicateurs. Cette modélisation explicite les invariants que l'implémentation doit garantir et fournit un langage rigoureux pour raisonner sur le matching et les performances.

5.1 Ensembles fondamentaux

Le système est décrit par les ensembles d'entités suivants, tous identifiés par des UUID :

T (Tenants), U (Users), A (Actors), B (Businesses), S (ServiceCatalogues),
 R (Resources), Po (Portfolios), SR (ServiceRequests), I (Invoices), BR (BusinessRules).

L'ensemble des devises supportées est fini :

$$C = \{XAF, XOF, NGN, KES, GHS, USD, EUR, GBP\}.$$

5.1.1 Décomposition des acteurs

$$A = A_c \cup A_p, \quad A_c \cap A_p \neq \emptyset,$$

où A_c (consommateurs) et A_p (fournisseurs) peuvent se recouper : un même acteur peut à la fois consommer et fournir des services.

5.1.2 Attributs des entités principales

$$t = (id_T, name, domain, isActive, createdAt) \in T,$$

$$u = (id_U, tenant, email, roles, isActive, \dots) \in U,$$

$$a = (id_A, user, role, bio, isActive) \in A,$$

$$sr = (id_{SR}, tenant, consumer, provider, catalogue, status, traceId, \dots) \in SR,$$

$$i = (id_I, serviceRequest, amount, currency, status, issuedAt, paidAt) \in I.$$

5.2 Variables de décision et fonction objectif

5.2.1 Variables

Variable binaire de matching :

$$x_{a_c, a_p, b} \in \{0, 1\}, \quad \forall (a_c, a_p, b) \in A_c \times A_p \times B,$$

valant 1 si le consommateur a_c est mis en relation avec le fournisseur a_p pour le métier b .

Variable d'état d'une demande :

$$\sigma_{sr} \in \Sigma_{SR} = \{\text{PENDING}, \text{ACCEPTED}, \text{IN_PROGRESS}, \text{FULFILLED}, \text{CANCELLED}\}.$$

5.2.2 Objectif multi-critère

On cherche à maximiser le nombre de transactions réussies et le chiffre d'affaires, tout en minimisant la durée de traitement :

$$\begin{aligned} \max \quad & \alpha \sum_{sr \in SR} \mathbb{1}_{\sigma_{sr}=\text{FULFILLED}} + \beta \sum_{i \in I} \text{amount}(i) \mathbb{1}_{\sigma_i=\text{PAID}} - \gamma \sum_{sr \in SR} \text{duration}(sr), \\ \text{avec } & \alpha + \beta + \gamma = 1, \quad \alpha, \beta, \gamma \geq 0. \end{aligned}$$

5.3 Contraintes du système

5.3.1 Contraintes de cardinalité

$$\forall u_1, u_2 \in U, u_1 \neq u_2 : u_1.\text{email} \neq u_2.\text{email} \quad (\text{unicité de l'email}) \quad (5.1)$$

$$\forall a \in A, \exists! po \in Po : po.\text{actor} = a \quad (\text{Actor} \rightarrow \text{Portfolio}, 1:1) \quad (5.2)$$

$$\forall po \in Po : po.\text{businesses} \subseteq B \quad (\text{Portfolio} \rightarrow \text{Business}, N:M) \quad (5.3)$$

$$\forall sr \in SR, \exists! i \in I : i.\text{serviceRequest} = sr \quad (\text{ServiceRequest} \rightarrow \text{Invoice}, 1:1) \quad (5.4)$$

5.3.2 Contraintes métier

Validité d'une demande :

$$\text{valid}(sr) \iff \begin{cases} sr.\text{consumer.role} = \text{CONSUMER}, \\ sr.\text{provider.role} = \text{PROVIDER}, \\ sr.\text{provider.isActive} = \text{vrai}. \end{cases}$$

Compatibilité de matching :

$$\text{compatible}(a_c, a_p, b) \iff \exists po \in Po : (po.\text{actor} = a_p \wedge b \in po.\text{businesses}).$$

Contrainte de devise :

$$\forall i \in I : i.\text{currency} \in C, \quad \forall s \in S : s.\text{currency} \in C.$$

Contrainte d'isolation multi-tenant (invariant de sécurité) :

$$\forall sr \in SR : sr.tenant = \tau \implies (sr.consumer.user.tenant = \tau \wedge sr.catalogue.business.tenant = \tau).$$

5.3.3 Contrainte de workflow : machine à états

Soit $\delta_{SR} : \Sigma_{SR} \times \mathcal{A} \rightarrow \Sigma_{SR}$ la fonction de transition, avec $\mathcal{A} = \{\text{create, accept, start, fulfill, cancel}\}$. Les transitions valides sont données au tableau 5.1.

Table 5.1: *Transitions valides de la machine à états ServiceRequest*

État courant	Action	État suivant
$\emptyset$	create	PENDING
PENDING	accept	ACCEPTED
PENDING	cancel	CANCELLED
ACCEPTED	start	IN_PROGRESS
ACCEPTED	cancel	CANCELLED
IN_PROGRESS	fulfill	FULFILLED
IN_PROGRESS	cancel	CANCELLED

5.4 Algorithmes et complexité

5.4.1 Recherche des fournisseurs d'un métier

```

1 Entree : b (business cible)
2 Sortie : ensemble des providers compatibles
3 1: Resultat <- vide
4 2: pour chaque po dans Po :
5 3:   si b appartient a po.businesses :
6 4:     a <- po.actor
7 5:     si a.role = PROVIDER et a.isActive : Resultat <- Resultat U {a}
8 6: retourner Resultat

```

Listing 5.1: *FindProvidersByBusiness*

Complexité : $O(|Po|)$ dans le pire cas, $O(1)$ amorti avec un index sur les métiers du portfolio.

5.4.2 Recherche par filtres combinés

Pour un ensemble de filtres $F = \{f_1, \dots, f_n\}$, le résultat est l'intersection

$$\text{result}(F) = \bigcap_{f \in F} \text{applyFilter}(f, \text{Univers}),$$

optimisée en appliquant d'abord le filtre le plus sélectif :

$$\text{selectivity}(f) = \frac{|\text{applyFilter}(f, \text{Univers})|}{|\text{Univers}|}, \quad \text{ordre croissant de selectivity.}$$

5.5 Indicateurs et métriques (KPIs)

Sur une période $[t_1, t_2]$, on définit :

$$T_{\text{conv}} = \frac{N_{\text{fulfilled}}}{N_{\text{requests}}} \times 100\%, \quad PA = \frac{CA}{N_{\text{paid}}},$$

où le chiffre d'affaires est

$$CA(t_1, t_2) = \sum_{\substack{i \in I : \sigma_i = \text{PAID} \\ i.\text{paidAt} \in [t_1, t_2]}} i.\text{amount},$$

et le temps moyen de réalisation

$$T_{\text{compl}} = \frac{1}{N_{\text{fulfilled}}} \sum_{sr \in SR : \sigma_{sr} = \text{FULFILLED}} (sr.\text{fulfilledAt} - sr.\text{requestedAt}).$$

La part d'un métier b dans le chiffre d'affaires s'écrit $\text{Share}_b = CA_b / CA \times 100\%$, et la croissance mensuelle $\text{Growth}_{\text{MoM}}(t) = (CA(t) - CA(t-1)) / CA(t-1) \times 100\%$. Ces indicateurs sont calculés par le service d'analytique (chapitre 10).

CHAPITRE 6

Conception architecturale

La pile protocolaire en cinq couches qui structure la lecture de BCaaS a été présentée au chapitre 3 (figure 3.2). Le présent chapitre se concentre sur les choix d'architecture logicielle qui la concrétisent : l'architecture hexagonale, le monolithe modulaire et l'organisation en packages.

6.1 Architecture hexagonale et DDD

BCaaS adopte une architecture hexagonale (ports et adaptateurs) combinée au Domain-Driven Design. Le domaine métier — entités, règles, machines à états — est isolé au centre ; les détails techniques (persistance JPA, exposition REST, messagerie) sont relégués aux adaptateurs périphériques. Cette organisation garantit que le cœur métier ne dépend d'aucune technologie particulière, condition d'une évolution ultérieure sereine.

Concrètement, le flux d'une requête traverse les couches applicatives suivantes :

Controller (REST) → Service (domaine) → Repository (JPA) → Base de données,

les DTO assurant le découplage entre la représentation exposée et les entités persistées.

6.2 Monolithe modulaire « microservices-ready »

Conformément aux contraintes du sujet, BCaaS est un **monolithe modulaire** : les modules métier (acteurs, métiers, catalogues, demandes, factures, audit, analytique) sont organisés en packages cohérents et communiquent par appels en mémoire. Ce choix simplifie le déploiement et la coordination initiale tout en préservant des frontières nettes : les modules pourront, à terme, être extraits en microservices déployés indépendamment, sans réécrire le domaine (perspective détaillée au chapitre 14).

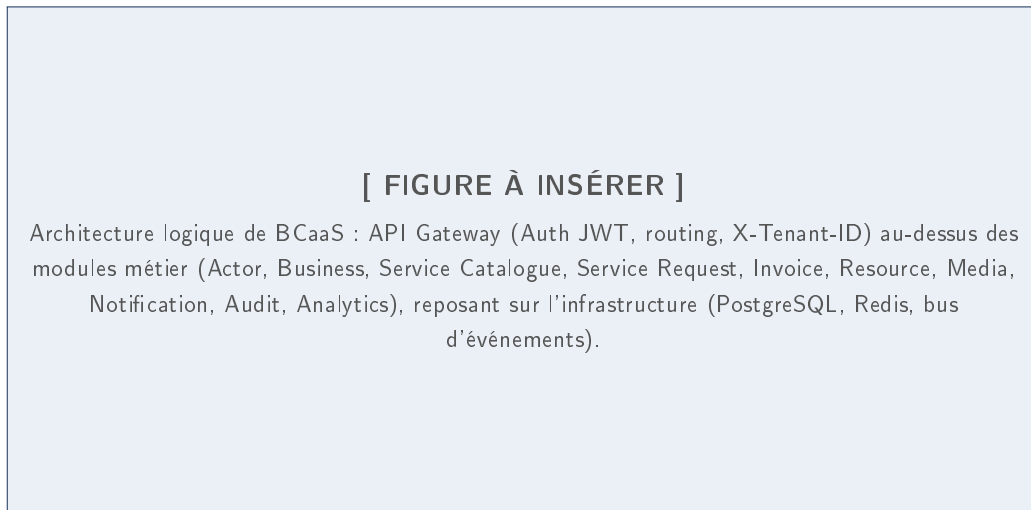


Figure 6.1: *Architecture logique de BCaaS*

6.3 Diagramme de packages

L'organisation en packages reflète les domaines fonctionnels : authentification, gestion des acteurs, gestion des métiers, gestion des services, gestion des médias, gestion des ressources et gestion des factures. Chaque package porte des responsabilités bien délimitées, ce qui facilite l'évolution indépendante des règles métier et des mécanismes techniques.

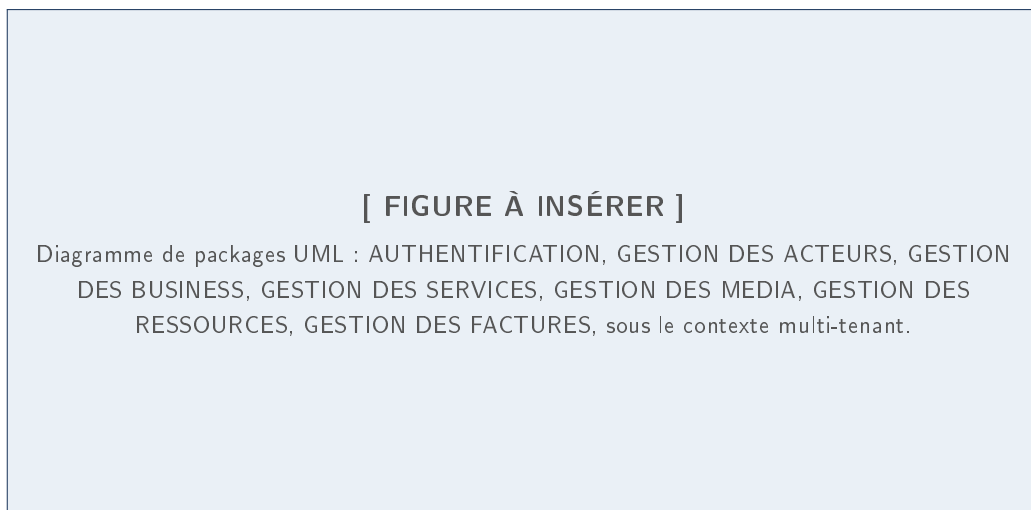


Figure 6.2: *Diagramme de packages de l'architecture du système*

6.4 Diagramme de déploiement

Sur le plan physique, l'API Spring Boot et sa base PostgreSQL sont déployées sur Render ; le frontend Next.js et la documentation sont déployés sur Vercel. En environnement local, un `docker-compose` complet ajoute Redis et le bus d'événements (chapitre 8).

[FIGURE À INSÉRER]

Diagramme de déploiement UML : nœud « Render » (conteneur API Spring Boot + PostgreSQL), nœud « Vercel » (frontend Next.js + docs), nœud client (navigateur) ; communications HTTPS/REST.

Figure 6.3: *Diagramme de déploiement de BCaaS*

CHAPITRE 7

Conception détaillée des données

Le défi central de BCaaS est de modéliser des entités métier hétérogènes (médecin, livre, appartement, produit agricole, etc.) avec un schéma de données unique et générique. Nous adoptons une modélisation Merise (MCD, MLD) complétée par un diagramme de classes UML.

7.1 Modèle Conceptuel de Données (MCD)

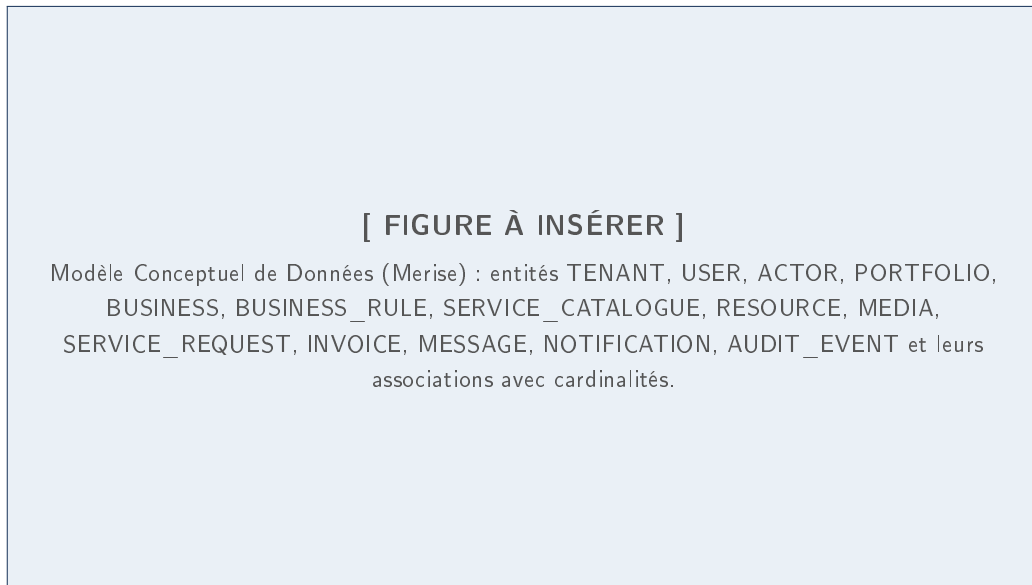


Figure 7.1: *Modèle Conceptuel de Données de BCaaS*

7.2 Dictionnaire des entités

Le domaine compte quinze entités (plus l'énumération **SupportedCurrency** et la table de jonction **portfolio_businesses**). Le tableau 7.1 en récapitule la structure.

Table 7.1: *Entités du modèle de données — Business Core*

Entité (table)	Attributs principaux et relations
Tenant (tenants)	<code>id</code> , <code>name</code> (unique), <code>domain</code> , <code>description</code> , <code>isActive</code> , <code>createdAt</code> . Racine du multi-tenant.
User (users)	<code>id</code> , <code>firstName</code> , <code>lastName</code> , <code>email</code> (unique), <code>password</code> , <code>phone</code> , <code>country</code> , <code>roles</code> (Set), <code>isActive</code> , <code>createdAt</code> ; <code>ManyToOne</code> → <code>Tenant</code> .
Actor (actors)	<code>id</code> , <code>role</code> , <code>bio</code> , <code>isActive</code> , <code>createdAt</code> ; <code>ManyToOne</code> → <code>User</code> .
Business (businesses)	<code>id</code> , <code>name</code> , <code>domain</code> , <code>description</code> , <code>typeOfInvolvedActors</code> , <code>requiredJobProfile</code> , <code>createdAt</code> ; <code>ManyToOne</code> → <code>Tenant</code> .
BusinessRule (business_rules)	<code>id</code> , <code>ruleKey</code> , <code>ruleValue</code> , <code>description</code> , <code>createdAt</code> ; <code>ManyToOne</code> → <code>Business</code> .
ServiceCatalogue (service_catalogues)	<code>id</code> , <code>name</code> , <code>description</code> , <code>basePrice</code> , <code>currency</code> , <code>isAvailable</code> , <code>createdAt</code> ; <code>ManyToOne</code> → <code>Business</code> .
Resource (resources)	<code>id</code> , <code>name</code> , <code>type</code> , <code>quantity</code> , <code>description</code> , <code>createdAt</code> ; <code>ManyToOne</code> → <code>Business</code> .
Media (media)	<code>id</code> , <code>type</code> , <code>url</code> , <code>label</code> , <code>createdAt</code> ; <code>ManyToOne</code> → <code>Business</code> .
Portfolio (portfolios)	<code>id</code> , <code>title</code> , <code>description</code> , <code>createdAt</code> ; <code>OneToOne</code> → <code>Actor</code> ; <code>ManyToMany</code> → <code>Business</code> (<code>portfolio_businesses</code>).
ServiceRequest (service_requests)	<code>id</code> , <code>serviceName</code> , <code>description</code> , <code>status</code> , <code>traceId</code> , <code>correlationId</code> , 5 timestamps ; <code>ManyToOne</code> → <code>Tenant</code> , consumer Actor, provider Actor, <code>ServiceCatalogue</code> .
Invoice (invoices)	<code>id</code> , <code>amount</code> , <code>currency</code> , <code>status</code> , <code>issuedAt</code> , <code>paidAt</code> ; <code>OneToOne</code> → <code>ServiceRequest</code> (unique).
Message (messages)	<code>id</code> , <code>content</code> , <code>isRead</code> , <code>createdAt</code> ; <code>ManyToOne</code> → <code>ServiceRequest</code> , sender Actor, <code>Tenant</code> .
Notification (notifications)	<code>id</code> , <code>title</code> , <code>message</code> , <code>isRead</code> , <code>createdAt</code> ; <code>ManyToOne</code> → <code>User</code> , <code>Tenant</code> .
AuditEvent (audit_events)	<code>id</code> , <code>entityType</code> , <code>action</code> , <code>previousStatus</code> , <code>newStatus</code> , <code>ipAddress</code> , <code>userAgent</code> , <code>metadata</code> (JSON), <code>tenantId</code> , <code>actorId</code> , <code>entityId</code> , <code>traceId</code> (UUID bruts).

Toutes les entités utilisent une génération d'identifiant `GenerationType.UUID` et un `@PrePersist` initialisant `createdAt`.

7.3 Modèle Logique de Données (MLD)

Le passage au modèle relationnel traduit les associations par des clés étrangères. En particulier, l'isolation multi-tenant repose sur une colonne `tenant_id` (UUID, clé étrangère vers `tenants`) portée par les tables racines du tenant (`users`, `businesses`, `service_requests`, `messages`, `notifications`, `audit_events`). L'entité `Actor` n'a pas de `tenant_id` direct : elle hérite du contexte tenant via son `User`.

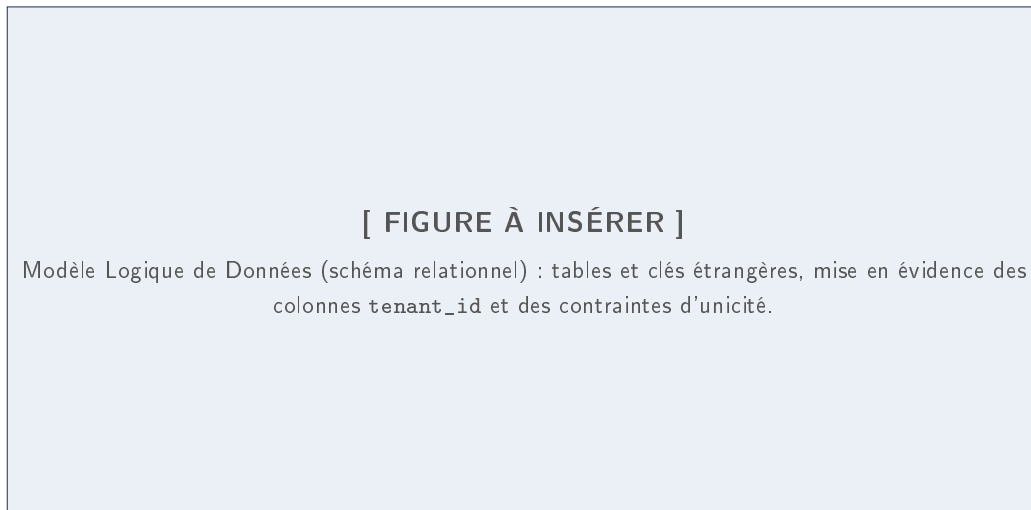


Figure 7.2: *Modèle Logique de Données de BCaaS*

7.4 Diagramme de classes métier

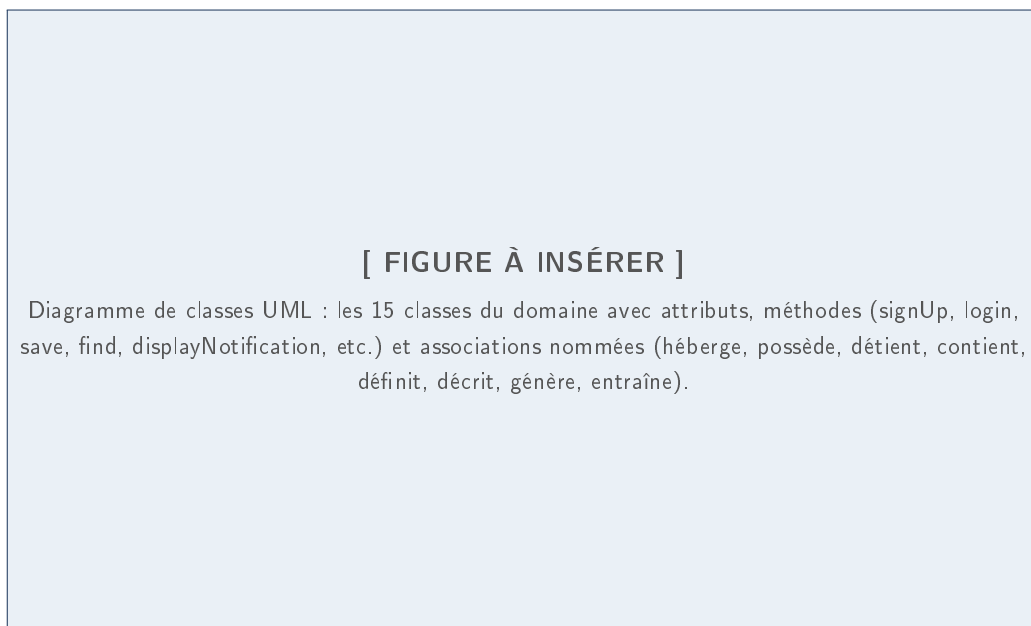


Figure 7.3: *Diagramme de classes métier de BCaaS*

7.5 Contraintes d'intégrité

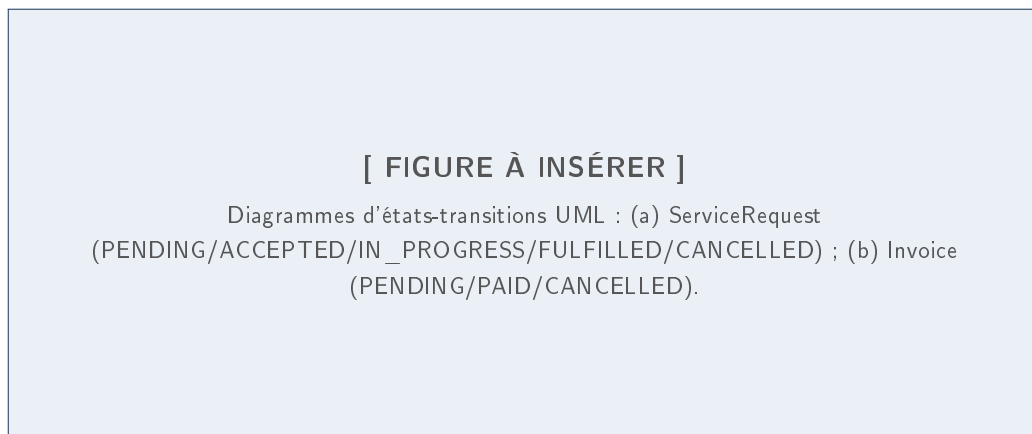
Le tableau 7.2 liste les principales contraintes d'intégrité référentielle et d'unicité.

Table 7.2: Principales contraintes d'intégrité

Table	Type	Description
tenants	Unique	Nom de tenant unique
users	Unique	Email unique
users	FK	tenant_id → tenants
portfolios	Unique + FK	Un portfolio par acteur
service_requests	FK	consumer, provider, catalogue, tenant existants
invoices	Unique + FK	Une facture par demande de service
messages, notifications, audit_events	FK (NOT NULL)	tenant_id obligatoire

7.6 Machines à états

Deux entités portent une machine à états explicite. La **ServiceRequest** suit le cycle **PENDING** → **ACCEPTED** → **IN_PROGRESS** → **FULFILLED**, avec **CANCELLED** accessible depuis tout état non terminal ; chaque transition horodate un champ dédié (**requestedAt**, **acceptedAt**, **startedAt**, **fulfilledAt**, **cancelledAt**). La transition **fulfill** déclenche la génération automatique de l'**Invoice**. La facture suit quant à elle **PENDING** → **PAID** ou **CANCELLED**.

**Figure 7.4:** Machines à états de *ServiceRequest* et *Invoice*

CHAPITRE 8

Implémentation technique

8.1 Pile technologique

Le tableau 8.1 présente la pile effectivement utilisée. Conformément au cadrage MVP retenu, les versions et composants y sont indiqués tels qu'ils sont réellement mis en œuvre ; les briques avancées non câblées sont reportées au chapitre 14.

Table 8.1: *Pile technologique de BCaaS*

Composant	Technologie	Version
Framework backend	Spring Boot (servlet, MVC)	3.5.x
Langage	Java	21 LTS
Base de données	PostgreSQL	16
Migrations	Liquibase	4.x
Sécurité	Spring Security + JWT (jjwt)	0.12.6
Documentation API	SpringDoc OpenAPI	2.8.6
Cache / pub-sub	Redis	7
Bus d'événements	Apache Kafka (infra)	7.6 (image)
Build	Maven	3.9+
Tests	JUnit 5 + Mockito + AssertJ	152 tests
Conteneurisation	Docker + Docker Compose	—
Frontend	Next.js / React / TypeScript	16 / 19

8.2 Structure du projet

Le projet suit l'architecture hexagonale combinée au DDD. L'arborescence backend est organisée par responsabilités techniques au sein du package `com.bizcore.bizcore_backend`.

```
1 src/main/java/com/bizcore/bizcore_backend/
2   +-- auth/           # Authentification (register, login, roles)
3   +-- config/         # Configurations (Kafka, Redis, Security, cache)
4   +-- controller/     # Controllers REST
5   +-- domain/         # Entites JPA (15) + enum SupportedCurrency
6   +-- dto/            # Objets de transfert
7   +-- exception/      # GlobalExceptionHandler, ApiError
8   +-- listener/       # Listener Kafka (consommateur)
9   +-- repository/     # Repositories Spring Data (tenant-aware)
10  +-- security/        # JwtService, JwtAuthFilter, TenantFilter, TenantContext
```

```

11  +-- service/          # Services metier
12  src/main/resources/
13  +-- application.properties
14  +-- db/changelog/ # Migrations Liquibase
15  src/test/            # 16 classes, 152 tests (JUnit 5 + Mockito)

```

Listing 8.1: Structure du projet backend

8.3 Multi-tenant et isolation

L'isolation repose sur le modèle « schéma partagé, colonne `tenant_id` » et sur trois pièces coordonnées.

1. **TenantContext** — un `ThreadLocal`. Chaque requête HTTP dispose d'une copie isolée du tenant actif, stockée dans un `ThreadLocal<UUID>`, ce qui évite toute fuite entre requêtes concurrentes partageant le même pool de threads.

2. **TenantFilter** — **extraction depuis le JWT**. Filtre `OncePerRequestFilter` d'ordre 2 (après le filtre d'authentification), il lit le claim `tenantId` du JWT et le place dans le contexte, puis le purge dans un bloc `finally`.

```

1  String tenantIdStr = jwtService.extractTenantId(jwt);    // claim "tenantId"
2  TenantContext.setTenantId(UUID.fromString(tenantIdStr));
3  try {
4      filterChain.doFilter(request, response);
5  } finally {
6      TenantContext.clear();    // pas de fuite inter-tenant
7  }

```

Listing 8.2: Extrait simplifié de `TenantFilter`

3. **Injection du claim à la connexion**. À l'authentification, `JwtService` embarque `tenantId` et `tenantName` dans le token. Le tenant est ainsi dérivé de l'utilisateur (colonne `users.tenant_id`) et relu à chaque requête sans appel supplémentaire à la base. L'isolation est appliquée dans la couche service via des requêtes *tenant-aware* de type `findAllByTenantId(...)`.

Garantie d'isolation et limite assumée

Le `tenant_id` circule dans le contexte d'exécution et conditionne les requêtes des services principaux ; un générateur de clés de cache préfixe par tenant. **Limite assumée** : l'isolation n'est pas une barrière globale (ni filtre Hibernate `@Filter`, ni *Row-Level Security* PostgreSQL) ; elle repose sur la discipline de chaque méthode de service. Le renforcement de cette barrière est traité au chapitre 14.

8.4 Sécurité : JWT et RBAC

L'authentification repose sur un couple email / mot de passe chiffré par BCrypt. Le token JWT, signé en HMAC-SHA, porte les rôles et le `tenantId`. Le contrôle d'accès est basé sur les rôles (`USER`, `ADMIN`, `PROVIDER`, `CONSUMER`), un même utilisateur pouvant cumuler plusieurs rôles. Spring Security joue ici le rôle d'un pare-feu applicatif filtrant les requêtes selon l'analogie du tableau 3.1.

8.5 Persistance et migrations

Le schéma est géré par Liquibase, garantissant des migrations versionnées et reproductibles (création du schéma, ajout des tables `notifications`, `messages`, `audit_events`, colonnes `tenant_id` contraintes par clé étrangère et indexées). Le paramètre `ddl-auto=none` laisse Liquibase seul maître du schéma en exécution.

8.6 Temps réel et événementiel

La messagerie instantanée sur les demandes de service est réalisée par **Redis publish/subscribe** : le service de messages publie sur un canal `messages:<requestId>`. Un **bus d'événements Kafka** est configuré (`KafkaTemplate` côté producteur, `@KafkaListener` sur le topic `bizcore.events` côté consommateur) ; l'infrastructure est posée et le consommateur opérationnel, la *production* d'événements restant à câbler (perspective au chapitre 14). Le cache Redis est de même configuré (gestionnaire de cache, TTL par espace) et prêt à être activé par annotations.

8.7 Documentation et conteneurisation

L'API est documentée automatiquement via SpringDoc OpenAPI et explorable par Swagger UI (annexe A.3). Un `docker-compose` décrit l'environnement local complet (application, PostgreSQL, Redis, Kafka avec *healthchecks* et dépendances ordonnancées) ; en production, l'API et sa base s'exécutent sur Render.

CHAPITRE 9

Protocole d'échange pour un cas métier concret

Pour illustrer la « pensée protocolaire » du projet, ce chapitre formalise l'échange d'un cas métier complet — la création d'un **Business** puis l'ajout d'un service à son catalogue — comme on spécifierait un protocole réseau : en-têtes obligatoires, payload, réponses, codes d'erreur et sémantique de transport.

9.1 Cas choisi : création d'un Business et ajout au catalogue

Ce cas correspond à deux requêtes distinctes mais liées, typiques de l'enregistrement d'une nouvelle activité (auto-école, cabinet de conseil, etc.) dans un tenant.

9.2 Protocole : création d'un Business

Endpoint : POST /api/businesses

En-têtes obligatoires :

- **Authorization**: Bearer <JWT> — authentification (analogie : ticket de session) ; le claim `tenantId` joue le rôle de l'adresse réseau.

```
1 {  
2   "name": "Auto-Ecole Centrale",  
3   "domain": "driving_school",  
4   "description": "Numero 1 dans la formation auto",  
5   "typeOfInvolvedActors": ["instructor", "student"],  
6   "requiredJobProfile": "moniteur"  
7 }
```

Listing 9.1: Corps de la requête (payload)

```
1 {  
2   "id": "550e8400-e29b-41d4-a716-446655440000",  
3   "name": "Auto-Ecole Centrale",  
4   "domain": "driving_school",  
5   "createdAt": "2025-05-04T10:00:00Z"  
6 }
```

Listing 9.2: Réponse (succès 201 Created)

Codes d'erreur (analogie : messages ICMP).

- 400 Bad Request : payload invalide ;
- 401 Unauthorized : JWT manquant ou invalide ;
- 403 Forbidden : rôle insuffisant ;
- 404 Not Found : ressource inexistante ;
- 409 Conflict : ressource déjà existante.

9.3 Protocole : ajout au catalogue de services

Endpoint : POST /api/service-catalogues/business/{businessId}

En-têtes : identiques, plus une corrélation optionnelle.

```
1 {  
2   "name": "Cours de conduite",  
3   "description": "Formation theorique et pratique de 3 mois",  
4   "basePrice": 150000,  
5   "currency": "XAF",  
6   "isAvailable": true  
7 }
```

Listing 9.3: Corps de la requête

Sémantique du transport.

- **Mode** : synchrone (REST) — analogie TCP (fiable, ordonné) ;
- **Timeout** : de l'ordre de 30 secondes ;
- **Idempotence** : non (chaque POST crée une nouvelle ressource) ;
- **Retry** : recommandé avec *backoff* exponentiel (au plus 3 tentatives).

9.4 Diagramme de séquence

[FIGURE À INSÉRER]

Diagramme de séquence UML du cas « création Business + ajout au catalogue » : acteurs Provider, API Gateway, TenantFilter, BusinessService, ServiceCatalogueService, base de données ; messages REST et réponses 201, propagation du traceId.

Figure 9.1: *Diagramme de séquence du cas métier « Business + Catalogue »*

CHAPITRE 10

Traçabilité et observabilité

L'observabilité de BCaaS s'appuie sur des éléments réellement implémentés — une piste d'audit, des identifiants de trace propagés et un service d'analytique — et propose des extensions (métriques Prometheus, QoS différenciée) reprises au chapitre 14.

10.1 Piste d'audit

L'entité `AuditEvent` enregistre les événements significatifs du système : type d'entité concernée (`SERVICE_REQUEST`, `INVOICE`, `ACTOR`, `BUSINESS`, `SERVICE_CATALOGUE`), action (`CREATED`, `STATUS_CHANGED`, `UPDATED`, `DELETED`), statut précédent et nouveau, adresse IP, *user agent* et métadonnées au format JSON. Chaque événement porte les identifiants bruts `tenantId`, `actorId`, `entityId` et `traceId`. L'API d'audit permet d'interroger ces événements par demande de service, par acteur, par tenant et par trace.

10.2 Tracing : `traceId` et `correlationId`

Chaque `ServiceRequest` porte un `traceId` et un `correlationId` (UUID), qui jouent le rôle d'identifiant de flux dans un protocole réseau. Ils permettent de reconstituer le chemin complet d'une transaction et de corréler entre eux les événements d'audit associés à une même opération métier.

Analogie : identifiant de flux

Le `traceId` est analogue au numéro de séquence TCP ou à l'identifiant de circuit virtuel : généré à l'entrée du système, il est propagé jusqu'aux écritures en base et aux événements émis, rendant chaque transaction traçable de bout en bout.

10.3 Analytique et KPIs

Le service d'analytique calcule, par tenant, les indicateurs clés formalisés au chapitre 5 (tableau 10.1). L'endpoint `GET /api/analytics/tenants/{tenantId}/kpis` les expose.

Table 10.1: *Métriques de BCaaS et leur analogie réseau*

Métrique	Analogie réseau	Calcul
Taux de conversion	Taux de livraison de paquets	demandes finalisées / total
Latence moyenne	RTT (Round Trip Time)	temps moyen de traitement
Débit (req/s)	Bande passante	requêtes par minute
Taux d'erreur	Taux de perte	réponses 4xx/5xx / total
Panier moyen	Taille moyenne de payload	chiffre d'affaires / nb factures
Délai d'exécution	Délai de routage	temps requête → finalisation

10.4 Logs structurés (proposition)

Pour compléter l'observabilité, des logs structurés au format JSON sont proposés, intégrant systématiquement `tenant_id`, `trace_id`, `actor_id`, le service, l'opération, la durée et le code HTTP — afin d'alimenter à terme une chaîne de collecte et un tableau de bord de métriques.

CHAPITRE 11

Frontend et déploiement

11.1 Architecture du frontend

Le frontend est développé avec Next.js (App Router) et React en TypeScript. Il se structure en deux parties correspondant aux deux plans de l'architecture : un **site vitrine** (plan de données vu de l'extérieur) présentant l'approche protocolaire, la documentation et des guides ; et un **tableau de bord** d'administration et de gestion (proche du plan de contrôle) pour piloter demandes, acteurs, métiers, services et factures.

11.2 Choix techniques

Table 11.1: Technologies du frontend

Besoin	Technologie	Justification
Framework	Next.js 16 (App Router)	SSR/SSG, SEO, routage moderne
UI	React 19 + TypeScript	Composants typés, robustesse
Styling	Tailwind CSS 3	Cohérence et rapidité
Composants	Radix UI	Accessibilité
Animations / 3D	Framer Motion, react-three/fiber	Vitrine immersive
État global	Zustand	Simple et performant
Données serveur	TanStack Query	Cache et synchronisation API

11.3 Pages principales

Les routes réellement implémentées comprennent : la page d'accueil (/), une démonstration (/demo), un assistant conversationnel (/chat), la documentation (/docs, /docs/[id]), des guides (/guides, /guides/[slug]), et un tableau de bord (/dashboard) avec ses sous-pages : demandes, acteurs, métiers, factures, services et paramètres.

11.4 Déploiement

L'API Spring Boot et sa base PostgreSQL sont déployées sur **Render** (<https://bizcore-api.onrender.com>). Le frontend et la documentation sont déployés sur **Vercel** (<https://>

bizcore-liard.vercel.app). La documentation interactive de l'API (Swagger UI) reste accessible depuis l'API déployée.

11.5 Captures d'écran



Figure 11.1: *Page d'accueil du site vitrine*

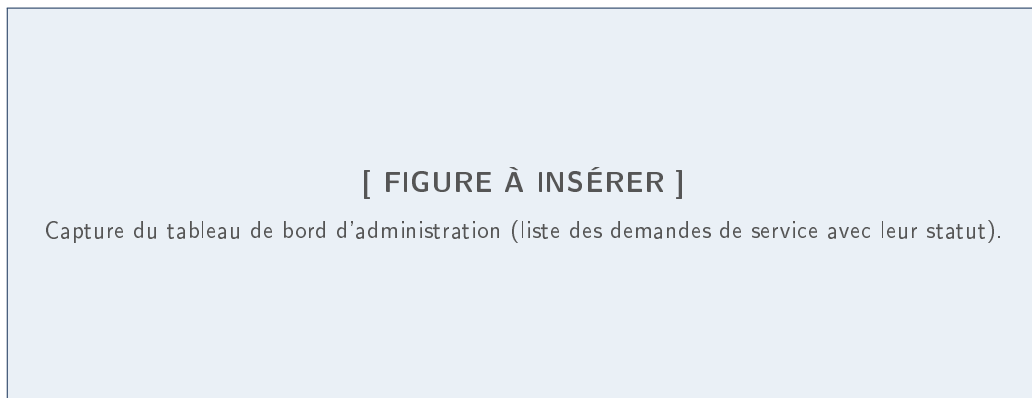


Figure 11.2: *Tableau de bord d'administration*

CHAPITRE 12

Tests et validation

12.1 Stratégie de test

La validation repose sur **16 classes de test totalisant 152 méthodes**, écrites avec JUnit 5, Mockito et AssertJ (via `spring-boot-starter-test`), complétés par `spring-security-test`. Les tests couvrent trois niveaux :

- **tests unitaires de services** : logique métier des services (Invoice, Business, Tenant, Actor, ServiceRequest, ServiceCatalogue, BusinessRule), ainsi que le contexte tenant (`TenantContextTest`) ;
- **tests de contrôleurs (WebMvc)** : Business, Invoice, Actor et ServiceRequest ;
- **tests de sécurité et d'intégration** : `JwtServiceTest`, `SecurityIntegrationTest`, `ServiceRequestIntegrationTest`.

L'environnement de test utilise une base H2 en mémoire (`ddl-auto=create-drop`, Liquibase désactivé), garantissant des tests rapides et isolés.

12.2 Critères d'acceptation

Le tableau 12.1 récapitule les critères d'acceptation et leur méthode de vérification.

Table 12.1: *Critères d'acceptation*

ID	Critère	Vérification
ACC-001	Les endpoints REST répondent correctement	Tests WebMvc + manuels
ACC-002	L'authentification JWT fonctionne	<code>JwtServiceTest</code> , <code>SecurityIntegrationTest</code>
ACC-003	Cycle CdS → FdS complet	<code>ServiceRequestIntegrationTest</code>
ACC-004	Génération automatique de la facture	Test de service + vérification BD
ACC-005	Isolation multi-tenant	<code>TenantContextTest</code>
ACC-006	Documentation Swagger à jour	Validation visuelle

12.3 Scénario de validation bout-en-bout

Le scénario nominal valide la chaîne complète de bout en bout :

1. inscription puis connexion (obtention du JWT portant le `tenantId`) ;
2. création d'un acteur, d'un portfolio et d'un métier ;
3. ajout d'un service au catalogue ;
4. émission d'une demande de service (CdS) ;
5. transitions `accept` → `start` → `fulfill`, avec génération automatique de la facture (ACK) ;
6. paiement de la facture.

Ce parcours exerce les machines à états, l'isolation tenant et la génération de facture, et constitue la démonstration de référence pour la soutenance.

Note sur la couverture. Aucun outil de mesure de couverture (JaCoCo) n'est intégré au projet ; les 152 tests constituent néanmoins une base de non-régression substantielle. L'ajout d'une mesure de couverture est cité au chapitre 14.

CHAPITRE 13

Gestion de projet

13.1 Méthodologie

Le projet a été conduit de façon incrémentale, organisée en sprints assortis de critères de priorité (P0 indispensable, P1 important). Chaque sprint livrait un incrément vérifiable et faisait l'objet d'une validation avant passage au suivant : modélisation Merise et UML, puis implémentation, puis tests automatisés et audits de code. Cette discipline a permis de détecter et corriger tôt les erreurs de modélisation (cardinalités, périmètre des entités).

13.2 Organisation des sprints

Table 13.1: *Découpage en sprints*

Sprint	Contenu
Sprint 1	Modélisation (MCD/MLD/UML), entités JPA, repositories, services de base, migrations Liquibase
Sprint 2	Sécurité JWT + RBAC, DTO, pagination, gestion globale des erreurs, devises
Sprint 3	Multi-tenant (TenantContext / TenantFilter), entités Message/Notification/Audit, analytique
Sprint 4	Frontend Next.js (vitrine + tableau de bord), documentation, déploiement (Render, Vercel)

13.3 Répartition des tâches et participation

Le tableau 13.2 présente la répartition de la contribution entre les membres du groupe. (*Les taux sont à compléter ; chacun est évalué sur 100.*)

Table 13.2: *Répartition de la participation*

Membre	Contributions principales	Participation
ETCHOME Chris		_____ / 100
[Membre 2 — Nom Prénom]		_____ / 100
[Membre 3 — Nom Prénom]		_____ / 100
[Membre 4 — Nom Prénom]		_____ / 100
[Membre 5 — Nom Prénom]		_____ / 100
[Membre 6 — Nom Prénom]		_____ / 100

13.4 Outils et gestion de version

Le code est versionné sous Git (dépôt GitHub), avec un flux de branches distinguant le code stable (**main**), l'intégration (**dev**) et les branches de fonctionnalités (**feat/***) ou de correction (**fix/***). La documentation technique est rédigée en Markdown et en L^AT_EX (le présent rapport).

CHAPITRE 14

Perspectives d'amélioration

Le MVP livré pose des fondations solides ; il ouvre néanmoins de nombreuses pistes d'amélioration, dont plusieurs correspondent aux contraintes technologiques avancées du sujet. Cette feuille de route est organisée par horizon.

14.1 Court terme : finalisation des briques posées

- **Câblage de l'événementiel Kafka.** L'infrastructure (producteur configuré, topic `bizcore.events`, consommateur de notifications) est en place ; il reste à publier effectivement les événements de domaine (changement de statut d'une demande, génération de facture) pour rendre la chaîne pleinement *event-driven*.
- **Activation du cache Redis.** Le gestionnaire de cache et les TTL sont définis ; l'ajout d'annotations `@Cacheable` / `@CacheEvict` sur les lectures fréquentes (règles métier, configuration tenant, catalogue) activerait le cache déjà préfixé par tenant.
- **Limitation de débit (rate limiting).** La dépendance Bucket4j est présente ; son intégration permettrait d'appliquer des quotas par tenant, prolongeant l'analogie de la bande passante et de la QoS.
- **Mesure de couverture de tests.** L'ajout de JaCoCo fournirait un indicateur chiffré de couverture.

14.2 Moyen terme : robustesse et qualité de service

- **Isolation multi-tenant renforcée.** Passer de la discipline applicative à une barrière systématique au moyen d'un filtre Hibernate `@Filter` ou de la *Row-Level Security* PostgreSQL, voire d'une isolation par schéma dédié pour les tenants premium.
- **QoS différenciée.** Introduire des niveaux de service inspirés de DiffServ (Bronze, Silver, Gold, Platinum) combinant priorité, latence cible et stratégie d'isolation.
- **Observabilité complète.** Exposer des métriques via Prometheus et un tableau de bord Grafana, et mettre en place des logs structurés agrégés.

14.3 Long terme : montée en charge et nouvelles contraintes

- **Architecture réactive.** Migration vers Spring WebFlux + R2DBC pour un modèle non bloquant, mieux adapté à une forte concurrence.

- **Géo-référencement natif.** Intégration de PostGIS et d'attributs de localisation pour les recherches géo-spatiales (*geolocation aware*).
- **Stockage d'objets.** Adoption de MinIO pour les médias, au lieu de la simple URL actuellement stockée.
- **Sécurité renforcée.** Double authentification (2FA) et migration vers OAuth 2.0.
- **Front offline-first et international.** Transformation du frontend en PWA installable, mode hors-ligne, et internationalisation (i18n) web et mobile.
- **Application mobile native.** Client React Native partageant l'API.
- **Microservices.** Extraction progressive des modules en services déployés indépendamment, le découpage actuel ayant été pensé pour cette évolution.
- **Marketplace d'instances.** À terme, permettre à des entreprises de déployer leur propre configuration BCaaS sans développement, comme socle d'un écosystème d'instances métier.

CHAPITRE 15

Conclusion générale

15.1 Bilan

Ce projet a démontré qu’une analogie rigoureuse entre l’ingénierie des protocoles réseaux et la conception d’une plateforme métier n’est pas une simple métaphore pédagogique : elle constitue une véritable méthode de conception. Les concepts de couches, d’encapsulation, de routage, d’adressage, de qualité de service et de séparation des plans de contrôle et de données ont guidé chaque décision architecturale de BCaaS.

Les réalisations principales sont :

- une grille de lecture protocolaire en cinq couches, formellement reliée au modèle OSI ;
- un modèle de données générique de quinze entités, configurable par tenant et implémenté sur PostgreSQL avec Liquibase ;
- une API REST d’environ quatre-vingt-cinq endpoints, documentée par OpenAPI et validée par cent cinquante-deux tests automatisés ;
- une authentification JWT multi-tenant avec contrôle d’accès basé sur les rôles, et une isolation par colonne `tenant_id` pilotée par un contexte `ThreadLocal` ;
- un protocole d’échange formalisé pour un cas métier concret ;
- une piste d’audit et un service d’analytique pour la traçabilité ;
- un frontend Next.js (site vitrine et tableau de bord), l’ensemble étant déployé sur le cloud (Render et Vercel).

15.2 Limites

Le périmètre livré est celui d’un MVP. Plusieurs briques avancées du sujet sont posées mais non pleinement câblées (événementiel Kafka, cache Redis par annotations, limitation de débit) ou restent à venir (réactif, géo-référencement, stockage objet, 2FA, PWA/offline, i18n, mobile natif). Ces limites, assumées et documentées, font l’objet de la feuille de route du chapitre 14.

15.3 Apports personnels

Ce projet a permis d’acquérir des compétences concrètes en architecture logicielle distribuée, en conception d’API REST professionnelle, en sécurité applicative (JWT, RBAC), en isolation multi-tenant et en modélisation de systèmes génériques. Au-delà des technologies, la démarche

de *pensée protocolaire* — définir les contrats avant le code, modéliser les échanges comme des messages — constitue un réflexe d'ingénieur directement transposable à tout projet de système d'information.

Bibliographie

- [1] Pr. Dr Ing. Thomas Djotio Ndié, *Cours d'Introduction aux Réseaux Locaux*, 3GI, ENSPY, 2025-2026.
- [2] ISO/IEC 7498-1, *Information technology — Open Systems Interconnection — Basic Reference Model*, ISO, 1994.
- [3] A. Cockburn, *Hexagonal Architecture (Ports and Adapters)*, 2005.
- [4] E. Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*, Addison-Wesley, 2003.
- [5] M. Jones, J. Bradley, N. Sakimura, *RFC 7519 — JSON Web Token (JWT)*, IETF, 2015.
- [6] Spring Boot Reference Documentation, <https://docs.spring.io/spring-boot/>, consulté en 2026.
- [7] SpringDoc OpenAPI Documentation, <https://springdoc.org>, consulté en 2025.
- [8] Next.js Documentation, <https://nextjs.org/docs>, consulté en 2026.
- [9] GeeksforGeeks, *Single-Tenant vs Multi-Tenant Architecture*, <https://www.geeksforgeeks.org/system-design/single-tenant-vs-multi-tenant-architecture/>, consulté en 2025.
- [10] Plateforme e-learning ENSPY, <https://elearningenspy.com/course/view.php?id=56>, consulté en 2025.

Glossaire

BaaS (Business as a Service)

Modèle fournissant des capacités métier génériques via une API standardisée.

Tenant

Organisation cliente disposant d'une isolation logique de ses données et configurations ; analogue d'un VLAN.

Multi-tenant

Architecture servant plusieurs tenants sur une infrastructure partagée, avec isolation de leurs données.

Actor

Rôle joué par un utilisateur dans le système : consommateur (CdS) et/ou fournisseur (FdS) de service.

CdS / FdS

Consommateur de Service (émetteur de la requête) / Fournisseur de Service (récepteur et exécutant).

Business

Métier ou profession référencé dans le système (domaine, règles, services, ressources).

BusinessRule

Règle métier sous forme de paire clé-valeur ; analogue du protocole.

ServiceRequest

Demande de service entre un CdS et un FdS ; analogue d'un message réseau, porteuse d'un `traceId`.

Invoice

Facture générée automatiquement à la finalisation ; analogue d'un accusé de réception (ACK).

Portfolio

Ensemble des métiers maîtrisés par un acteur (relation N:M avec Business).

FSM

Machine à états finis décrivant le cycle de vie d'une entité (ServiceRequest, Invoice).

JWT

Jeton signé portant l'identité, les rôles et le `tenantId` de l'utilisateur authentifié.

RBAC

Contrôle d'accès basé sur les rôles.

traceId / correlationId

Identifiants de flux permettant de tracer une transaction de bout en bout.

DTO

Objet de transfert de données découplant la représentation exposée des entités persistées.

Soft delete

Suppression logique préservant les données en base (`isActive = false`).

Control plane / Data plane

Plan de configuration (administration) / plan d'exécution (transactions), par analogie avec le SDN.

ANNEXE A

Annexes

A.1 Liste des endpoints REST

L'API expose environ 85 endpoints répartis sur 17 contrôleurs. Le tableau A.1 en donne une vue synthétique.

Table A.1: *Synthèse des endpoints REST*

Ressource	Endpoints
/api/auth	POST /register, POST /login, PATCH /users/{id}/roles, DELETE /users/{id}/roles
/api/tenants	GET, GET /{id}, POST /register, PUT /{id}, DELETE /{id}
/api/users	GET, GET /{id}, GET /email/{email}
/api/actors	GET, GET /{id}, GET /user/{userId}, GET /role/{role}, POST /user/{userId}, DELETE /{id}
/api/businesses	GET, POST, GET /{id}, GET /domain/{domain}, GET /search, PUT /{id}, DELETE /{id}
/api/business-rules	GET, GET /{id}, GET /business/{businessId}, POST /business/{businessId}, PUT /{id}, DELETE /{id}
/api/service-catalogues	GET, GET /{id}, GET /business/{businessId}, GET /available, GET /search, POST /business/{businessId}, PUT /{id}, DELETE /{id}
/api/service-requests	GET, GET /{id}, GET /consumer/{id}, GET /provider/{id}, GET /status/{status}, POST /consumer/{id}/provider/{id}/catalogue/{id}, PATCH /{id}/accept, /start, /fulfill, /cancel, DELETE /{id}
/api/invoices	GET, GET /{id}, GET /status/{status}, GET /service-request/{id}, POST /service-request/{id}, PATCH /{id}/pay, /cancel, DELETE /{id}
/.../messages	GET, POST, GET /unread-count, PATCH /read-all, DELETE /{messageId}
/api/notifications	GET, GET /unread-count, PATCH /{id}/read, PATCH /read-all, DELETE /{id}
/api/portfolios	GET, GET /{id}, GET /actor/{id}, POST /actor/{id}, PATCH /{id}/businesses/{bid}, DELETE /{id}/businesses/{bid}, DELETE /{id}
/api/resources	GET, GET /{id}, GET /business/{id}, POST /business/{id}, PUT /{id}, DELETE /{id}
/api/media	GET, GET /{id}, GET /business/{id}, GET /type/{type}, POST /business/{id}, PUT /{id}, DELETE /{id}
/api/currencies	GET, GET /validate/{code}
/api/analytics	GET /tenants/{tenantId}/kpis

Ressource	Endpoints
/api/audit	GET /service-requests/{id}, GET /actors/{id}, GET /trace/{traceId}, GET /tenants/{id}

A.2 Exemple de log structuré

```

1 {
2   "timestamp": "2025-05-04T10:05:23Z",
3   "level": "INFO",
4   "tenant_id": "550e8400-e29b-41d4-a716-446655440000",
5   "trace_id": "7e9a8b7c-6d5e-4f3a-2b1c-9d8e7f6a5b4c",
6   "actor_id": "actor_123",
7   "service": "BusinessService",
8   "operation": "createBusiness",
9   "duration_ms": 145,
10  "http_status": 201
11 }
```

Listing A.1: Log structuré au format JSON (proposition)

A.3 Capture Swagger UI

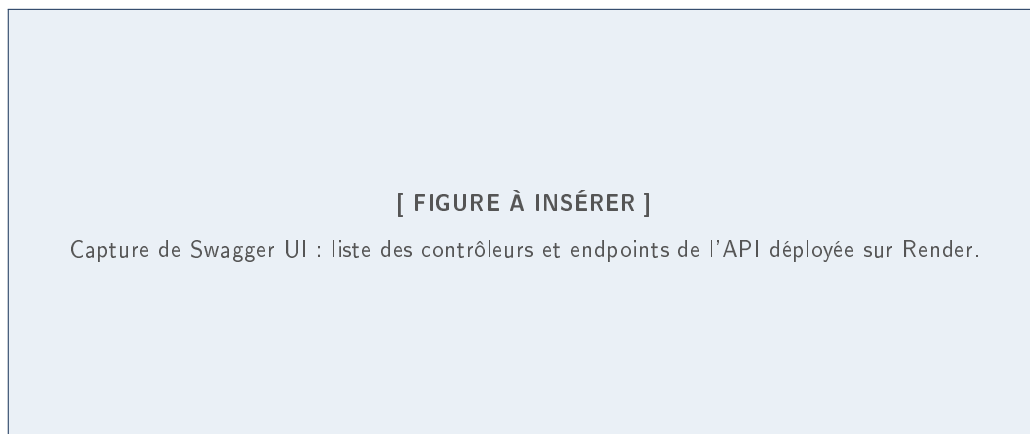


Figure A.1: Documentation interactive Swagger UI

A.4 Extraits de code significatifs

```

1 public final class TenantContext {
2     private static final ThreadLocal<UUID> CURRENT_TENANT = new ThreadLocal
3         <>();
4     public static void setTenantId(UUID tenantId) { CURRENT_TENANT.set(
5         tenantId); }
6     public static UUID getTenantId() { return CURRENT_TENANT.get(); }
7     public static void clear() { CURRENT_TENANT.remove(); }
```

```
6 }
```

Listing A.2: *TenantContext (ThreadLocal)*

```
1 UUID tenantId = TenantContext.getTenantId();  
2 if (tenantId != null) {  
3     return serviceRequestRepository.findAllByTenantId(tenantId, pageable);  
4 }
```

Listing A.3: *Lecture tenant-aware dans un service*